

TRM: Process Notes

What I actually got stuck on, in something closer to real time

Companion to the TRM case study, Fall 2025

This isn't the polished version. The case study is the curated account of what I did and why it mattered. This is closer to what the semester actually felt like week to week, cleaned up just enough to read, but not smoothed over. Some of this is pulled directly from my own working notes at the time.

Clerk auth, and not really understanding what a session token was

Early on I genuinely did not understand what `Clerk.session.getToken()` was giving me or why. I knew a token got a user access to the Express side, which had access to Supabase, but I could not have explained the chain if you'd asked me directly. I spent a while calling `getToken()` and getting a token that just did not work the way I expected, wrong encryption, wrong shape, before realizing I needed `getToken({ template: 'supabase-local' })` specifically, a named template that formats the token the way Supabase's row-level security actually expects.

It sounds small written out like that. At the time it was a solid chunk of confused Googling and re-reading Clerk's docs before it clicked that "a token" was not one universal thing, it was shaped differently depending on what was going to consume it.

See: Figure A, token flow diagram

Atomic design, in theory vs. actually placing a file

I thought I understood atomic design going in. Atoms, molecules, organisms, templates, pages, felt straightforward as a concept. Actually working inside the folder structure was a different problem entirely.

"the mainmenu.tsx is the only thing in molecules... everything else seems to just connect directly from organisms to atoms, skipping molecules level... why can some organisms skip the molecules level and grab directly from atoms and others go step by step"

That was a real question I had, taken from my own notes at the time. I did not have a clean answer for a while. What I eventually worked out, partly from the codebase and partly from asking, is that the levels are not a strict pipeline every piece of UI has to pass through, they are a vocabulary for how composed something is. A molecule only needs to exist if that specific grouping gets reused somewhere else. If it doesn't, going straight from atoms to an organism isn't a violation of the system, it's just what that particular piece needed. Same logic applies to templates, some pages have enough unique layout to earn one, some don't. Knowing the five words was not the same as knowing when each one applied.

See: Figure B, atomic design decision logic

Git, actually learning it instead of memorizing commands

This was probably the single biggest time sink of the semester, and also the thing I feel like I actually learned the most from by the end. The specific sequence I had to internalize, mostly through trial, error, and a lot of YouTube:

- Rebase a feature branch onto the latest release branch before opening a PR
- If the branch had multiple messy commits, squash them: `git rebase -i {commit hash}`, change `pick` to `s` for the commits being squashed, then rewrite the final commit message
- Run `npm run format` and `npm run lint` before pushing
- After a squash or rebase, a normal push usually gets rejected, since history changed. That's where `git push -f` comes in, and I remember being warned pretty directly that the `-f` flag is dangerous and not something to reach for casually
- A few times things got tangled enough that the fix was `git reset --hard {commit hash}` followed by `git fetch` and `git pull`, essentially throwing away my local mess and starting clean from origin

None of this was taught to me in a lecture. It was Marvin, our senior developer mentor, explaining it in the moment when something broke, plus a lot of independent reading afterward to make sure I actually understood why a command did what it did, not just which one to type.

See: Figure C, the rebase and squash sequence, redrawn from what I actually learned

Tooling ramp-up

Docker, Postman, Prettier, Tailwind, none of these were things I'd used seriously before this project. Tailwind in particular took a bit to click, I understood it was "a CSS library" in the

abstract but didn't have a real feel for why utility classes over traditional CSS until I'd actually built a few components with it and felt the consistency it forced across the team. Postman ended up being more useful than I expected once I actually needed to test endpoints in isolation instead of only through the running app.

The honest reflection

The thing I keep coming back to, and this is close to word for word what I wrote to myself partway through the semester:

"one thing that would have saved a bunch of time was learning and understanding the tools we were working with and the goals we were working towards before trying to work on the project, not just getting the gist of something, but actually understanding it"

I said basically the same thing about git and about atomic design separately, before realizing it was really one pattern repeating. I kept starting with just enough understanding to begin, then paying for the gaps later instead of front-loading the confusion. It worked, mostly, but it wasn't efficient, and it's the clearest, most specific thing I'd actually change if I did this again.